Python: exercises on branching

This notebook was generated in **pseudocode** mode.

Exercise: Positive, Negative, or Zero

Exercise Description

Write a program that determines whether a given number is positive, negative, or zero.

Use the number `num = -5` and store the result in variable `result` ("positive", "negative", or "zero").

Your solution

```
num = -5
# step 1: check if the number is greater than 0
    # step 2: assign "positive" to result
# step 3: otherwise, check if the number is less than 0
    # step 4: assign "negative" to result
```

```
# step 5: otherwise (the number is equal to 0)
    # step 6: assign "zero" to result
```

Test your solution

```
assert result == "negative"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Even or Odd Number

Exercise Description

Write a program that determines whether a given integer is even or odd.

Use the number `num = 7` and store the result in variable `result` ("even" or "odd").

Your solution

```
num = 7
# step 1: check if the number modulo 2 equals 0 (divisible by 2)
    # step 2: assign "even" to result
# step 3: otherwise (remainder is not 0)
    # step 4: assign "odd" to result
```

Test your solution

```
assert result == "odd"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Grade Evaluator

Exercise Description

Write a program that converts a numerical score (0-100) to a letter grade:

- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- 0-59: F

Use the score `score = 85` and store the letter grade in variable `result`.

Your solution

```
score = 85
# step 1: check if score is 90 or above for grade A
    # step 2: assign "A" to result
# step 3: otherwise, check if score is 80 or above for grade B
    # step 4: assign "B" to result
# step 5: otherwise, check if score is 70 or above for grade C
    # step 6: assign "C" to result
# step 7: otherwise, check if score is 60 or above for grade D
    # step 8: assign "D" to result
# step 9: otherwise (score is below 60)
    # step 10: assign "F" to result
```

Test your solution

```
assert result == "B"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Divisibility Checker

Exercise Description

Write a program that checks if a given number is divisible by both 3 and 5.

Use the number `num = 30` and store the result in variable `result` ("divisible by both" or "not divisible by both").

Your solution

```
num = 30
# step 1: check if number is divisible by 3 AND divisible by 5
    # step 2: use modulo operator to check if remainder is 0 for both 3 and 5
    # step 3: if both conditions are true, assign "divisible by both" to result
# step 4: otherwise
    # step 5: assign "not divisible by both" to result
```

Test your solution

```
assert result == "divisible by both"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Leap Year Checker

Exercise Description

Write a program that determines whether a given year is a leap year or not.

A leap year is:

- Divisible by 4 but not by 100, OR
- Divisible by 400

Use the year `year = 2024` and store the result in variable `result` ("leap year" or "not leap year").

Your solution

```
year = 2024
# step 1: check the complex leap year condition using logical operators
    # step 2: check if year is divisible by 4 AND not divisible by 100
    # step 3: OR check if year is divisible by 400
    # step 4: if either condition is true, assign "leap year" to result
# step 5: otherwise
    # step 6: assign "not leap year" to result
```

Test your solution

```
assert result == "leap year"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Vowel or Consonant

Exercise Description

Write a program that determines whether a given letter is a vowel or consonant.

Use the letter `letter = "A"` and store the result in variable `result` ("vowel" or "consonant").

Your solution

```
letter = "A"
# step 1: check if the letter is in the string of vowels (both lowercase and uppercas
    # step 2: use the 'in' operator to check if letter is in "aeiouAEIOU"
    # step 3: if true, assign "vowel" to result
# step 4: otherwise
    # step 5: assign "consonant" to result
```

Test your solution

```
assert result == "vowel"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Password Validator

Exercise Description

Write a program that validates a password. If the password is "password123", grant access, otherwise deny access.

Use the password `password = "password123"` and store the result in variable `result` ("Access granted" or "Access denied").

Your solution

```
password = "password123"
# step 1: check if the entered password equals the correct password "password123"
    # step 2: if passwords match, assign "Access granted" to result
# step 3: otherwise
    # step 4: assign "Access denied" to result
```

Test your solution

```
assert result == "Access granted"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Largest of Three Numbers

Exercise Description

Write a program that finds the largest of three given numbers.

Use the numbers `num1 = 15`, `num2 = 8`, and `num3 = 22`. Store the largest number in variable `result`.

Your solution

```
num1 = 15
num2 = 8
num3 = 22
# step 1: check if num1 is greater than or equal to both num2 and num3
    # step 2: assign num1 to result as the largest
# step 3: otherwise, check if num2 is greater than or equal to both num1 and num3
    # step 4: assign num2 to result as the largest
# step 5: otherwise (num3 must be the largest)
    # step 6: assign num3 to result as the largest
```

Test your solution

```
assert result == 22
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Day of the Week

Exercise Description

Write a program that converts a number (1-7) to the corresponding day of the week:

- 1: Monday
- 2: Tuesday
- 3: Wednesday
- 4: Thursday
- 5: Friday
- 6: Saturday
- 7: Sunday
- Any other number: "Invalid number"

Use the number `day = 5` and store the day name in variable `result`.

Your solution

```
day = 5
# step 1: check if day equals 1
    # step 2: assign "Monday" to result
# step 3: otherwise, check if day equals 2
    # step 4: assign "Tuesday" to result
# step 5: otherwise, check if day equals 3
    # step 6: assign "Wednesday" to result
# step 7: otherwise, check if day equals 4
    # step 8: assign "Thursday" to result
# step 9: otherwise, check if day equals 5
    # step 10: assign "Friday" to result
# step 11: otherwise, check if day equals 6
    # step 12: assign "Saturday" to result
# step 13: otherwise, check if day equals 7
    # step 14: assign "Sunday" to result
```

```
# step 15: otherwise (invalid number)
    # step 16: assign "Invalid number" to result
```

Test your solution

```
assert result == "Friday"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Simple Calculator

Exercise Description

Write a program that performs basic arithmetic operations (+, -, *, /) on two numbers based on the given operation.

Use `num1 = 10.0`, `num2 = 3.0`, and `operation = "+"`. Store the result in variable `result`. Handle division by zero with an error message.

Your solution

```python
num1 = 10.0
num2 = 3.0
operation = "+"
# step 1: check if operation is addition ("+")
    # step 2: perform addition and assign to result
# step 3: otherwise, check if operation is subtraction ("-")
    # step 4: perform subtraction and assign to result
# step 5: otherwise, check if operation is multiplication ("*")
    # step 6: perform multiplication and assign to result
# step 7: otherwise, check if operation is division ("/")
    # step 8: check if second number is not zero to avoid division by zero
        # step 9: perform division and assign to result
    # step 10: otherwise (if dividing by zero)
        # step 11: assign error message to result
# step 12: otherwise (invalid operation)
    # step 13: assign error message to result
```

Test your solution

```python
assert result == 13.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Triangle Type Classifier

Exercise Description

Write a program that determines the type of triangle based on the lengths of its three sides:

- Equilateral: All sides are equal
- Isosceles: Two sides are equal
- Scalene: No sides are equal

Use `side1 = 5.0`, `side2 = 5.0`, and `side3 = 8.0`. Store the triangle type in variable `result`.

Your solution

```python
side1 = 5.0
side2 = 5.0
side3 = 8.0
# step 1: check if all three sides are equal (equilateral triangle)
    # step 2: compare side1 == side2 == side3
    # step 3: if true, assign "equilateral" to result
# step 4: otherwise, check if any two sides are equal (isosceles triangle)
    # step 5: check if side1 == side2 OR side2 == side3 OR side1 == side3
```

```
    # step 6: if true, assign "isosceles" to result
# step 7: otherwise (no sides are equal)
    # step 8: assign "scalene" to result
```

Test your solution

```
assert result == "isosceles"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Maximum of Three Numbers

Exercise Description

Write a program that finds the maximum (largest) of three given numbers.

Use the numbers `num1 = 12.5`, `num2 = 8.3`, and `num3 = 15.7`. Store the maximum number in variable `result`.

Your solution

```
num1 = 12.5
num2 = 8.3
num3 = 15.7
# step 1: check if num1 is greater than or equal to both num2 and num3
    # step 2: assign num1 to result as the maximum
# step 3: otherwise, check if num2 is greater than or equal to both num1 and num3
    # step 4: assign num2 to result as the maximum
# step 5: otherwise (num3 must be the maximum)
    # step 6: assign num3 to result as the maximum
```

Test your solution

```
assert result == 15.7
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: BMI Calculator with Categories

Exercise Description

Calculate the Body Mass Index (BMI) and categorize it according to standard health categories:

- Underweight: BMI < 18.5
- Normal weight: $18.5 \leq$ BMI < 25
- Overweight: $25 \leq$ BMI < 30
- Obese: BMI ≥ 30

BMI formula: weight (kg) / (height (m))^2

Use `weight = 70` kg and `height = 1.75` m. Store the BMI category in variable `result`.

Your solution

```
weight = 70  # kg
height = 1.75  # m
# step 1: calculate BMI using the formula weight divided by height squared
# step 2: check if BMI is less than 18.5 for underweight category
    # step 3: assign "Underweight" to result
# step 4: otherwise, check if BMI is less than 25 for normal weight category
    # step 5: assign "Normal weight" to result
# step 6: otherwise, check if BMI is less than 30 for overweight category
    # step 7: assign "Overweight" to result
# step 8: otherwise (BMI is 30 or above)
    # step 9: assign "Obese" to result
```

Test your solution

```
assert result == "Normal weight"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Rock Paper Scissors Game

Exercise Description

Implement the logic for a Rock Paper Scissors game. Determine the winner based on the classic rules:

- Rock beats Scissors
- Scissors beats Paper
- Paper beats Rock
- Same choice results in a tie

Use `player1 = "rock"` and `player2 = "scissors"`. Store the result in variable `result` ("Player 1 wins", "Player 2 wins", or "It's a tie").

Your solution

```
player1 = "rock"
player2 = "scissors"
# step 1: check if both players chose the same option
    # step 2: assign "It's a tie" to result
# step 3: otherwise, check winning conditions for player 1
    # step 4: check if player1 is rock and player2 is scissors
    # step 5: OR check if player1 is scissors and player2 is paper
    # step 6: OR check if player1 is paper and player2 is rock
    # step 7: if any of these conditions is true, assign "Player 1 wins" to result
# step 8: otherwise (player 2 wins in all remaining cases)
    # step 9: assign "Player 2 wins" to result
```

Test your solution

```
assert result == "Player 1 wins"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Temperature Converter

Exercise Description

Create a temperature converter that converts between Celsius and Fahrenheit based on the conversion type:

- "C to F": Convert Celsius to Fahrenheit using F = (C × 9/5) + 32
- "F to C": Convert Fahrenheit to Celsius using C = (F - 32) × 5/9

Use `temperature = 25.0` and `conversion_type = "C to F"`. Store the converted temperature in variable `result`.

Your solution

```
temperature = 25.0
conversion_type = "C to F"
# step 1: check if conversion type is "C to F" (Celsius to Fahrenheit)
    # step 2: apply Fahrenheit formula: multiply temperature by 9/5 and add 32
    # step 3: assign the converted temperature to result
# step 4: otherwise, check if conversion type is "F to C" (Fahrenheit to Celsius)
    # step 5: apply Celsius formula: subtract 32 from temperature and multiply by 5/9
    # step 6: assign the converted temperature to result
```

Test your solution

```
assert result == 77.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Movie Ticket Price Calculator

Exercise Description

Calculate movie ticket price based on age and day of the week:

- Regular price: $12
- Child (under 12): $8
- Senior (65 and over): $10
- Tuesday discount: 20% off all tickets
- Weekend surcharge (Saturday/Sunday): $2 extra

Use `age = 25` and `day = "Tuesday"`. Store the ticket price in variable `result`.

Your solution

```python
age = 25
day = "Tuesday"
# step 1: determine base price based on age category
    # step 2: check if age is under 12 for child price
        # step 3: set base price to $8
    # step 4: otherwise, check if age is 65 or over for senior price
        # step 5: set base price to $10
    # step 6: otherwise (regular adult price)
        # step 7: set base price to $12
# step 8: apply day-specific adjustments
    # step 9: check if day is Tuesday for discount
        # step 10: apply 20% discount (multiply by 0.8)
    # step 11: otherwise, check if day is Saturday or Sunday for weekend surcharge
        # step 12: add $2 weekend surcharge
# step 13: assign final price to result
```

Test your solution

```python
assert result == 9.6  # $12 * 0.8 = $9.60
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise: Traffic Light Simulator

Exercise Description

Simulate a traffic light system that determines the action based on the current light color:

- "red": "Stop"
- "yellow": "Prepare to stop"
- "green": "Go"
- Any other color: "Invalid light color"

Use `light_color = "green"` and store the action in variable `result`.

Your solution

```
light_color = "green"
# step 1: check if light color is "red"
    # step 2: assign "Stop" to result
# step 3: otherwise, check if light color is "yellow"
    # step 4: assign "Prepare to stop" to result
# step 5: otherwise, check if light color is "green"
    # step 6: assign "Go" to result
```